Applicability of Neural Combinatorial Optimization: A Critical View

ANDONI I. GARMENDIA, JOSU CEBERIO, and ALEXANDER MENDIBURU, University of the Basque Country UPV/EHU, Donostia-San Sebastian, Spain

Neural Combinatorial Optimization has emerged as a new paradigm in the optimization area. It attempts to solve optimization problems by means of neural networks and reinforcement learning. In the past few years, due to their novelty and presumably good performance, many research papers have been published introducing new neural architectures for a variety of combinatorial problems. However, the incorporation of such models in the conventional optimization portfolio raises many questions related to their performance compared to other existing methods, such as exact algorithms, heuristics, or metaheuristics. This article aims to present a critical view of these new proposals, discussing their benefits and drawbacks with respect to the tools and algorithms already present in the optimization field. For this purpose, a comprehensive study is carried out to analyze the fundamental aspects of such methods, including performance, computational cost, transferability, and reusability of the trained model. Moreover, this discussion is accompanied by the design and validation of a new neural combinatorial optimization algorithm on two well-known combinatorial problems: the Linear Ordering Problem and the Permutation Flowshop Scheduling Problem. Finally, new directions for future work in the area of Neural Combinatorial Optimization algorithms are suggested.

CCS Concepts: • **Mathematics of computing → Combinatorial optimization**; *Graph algorithms*; • **Computing methodologies → Artificial intelligence;**

Additional Key Words and Phrases: Combinatorial optimization, reinforcement learning, Graph Neural Networks

ACM Reference Format:
Andoni I. Garmendia, Josu Ceberio, and Alexander Mendiburu. 2024. Applicability of Neural Combinatorial Optimization: A Critical View. *ACM Trans. Evol. Learn. Optim.* 4, 3, Article 15 (July 2024), 26 pages. https://doi.org/10.1145/3647644

1 INTRODUCTION

Combinatorial optimization deals with problems with an objective function that has to be maximized over a set of combinatorial alternatives [Papadimitriou and Steiglitz 1998]. The most naive way of obtaining the optimal solution is to list all the feasible solutions, evaluating them using the objective function, and selecting the optimal one. Nevertheless, the previous brute-force approach lacks practicality when the size of the problem is too large, as the time and computational resources needed to solve it grow exponentially with the problem size.

Authors' address: A. I. Garmendia, J. Ceberio, and A. Mendiburu, University of the Basque Country UPV/EHU, Manuel Lardizabal pasealekua, 1, Donostia-San Sebastian, Gipuzkoa, Spain, 20018; e-mails: andoni.irazusta@ehu.eus, josu.ceberio@ehu.eus, alexander.mendiburu@ehu.eus.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).
ACM 2688-3007/2024/07-ART15
https://doi.org/10.1145/3647644

Traditionally, **Combinatorial Optimization Problems (COPs)** have been addressed using either exact or heuristic methods. Exact methods provide an optimal solution if given sufficient time, but the time required can grow exponentially with the size of the problem. In contrast, heuristic methods do not guarantee an optimal solution but often produce reasonably good solutions within a limited time frame [Pearl 1984]. In general, the effectiveness of a heuristic method depends on its ability to identify and exploit the relevant information of the problem at hand. In this line, as a generalization of heuristic algorithms, **metaheuristics (MH)** introduce higher-level generic procedures that guide the search process, making them "general-purpose methods" [Blum and Roli 2003].

Even though plenty of work has been done in the development of MH algorithms, in the past decade the research in the area has reached a point of maturity, where the number of relevant algorithmic proposals has been reduced. At the same time, the expansion of **Deep Learning (DL)** techniques to the optimization domain has brought novel research lines. The recent success of DL in fields such as machine translation [Cho et al. 2014], biology [Jumper et al. 2021], or board games [Schrittwieser et al. 2020] has not only attracted the attention of many machine learning practitioners into the optimization field, but has also captured the interest of the optimization community regarding the possible use cases of DL.

Although the first attempts to include **Neural Networks (NN)** dates back to the '80s [Hopfield and Tank 1985], this research line has not attracted the interest of the community until recently. In fact, in the past few decades, NNs have become an interesting alternative, due to the increase in computational capacity and the development of complex NN models, which enable the design of competitive algorithms. Recent reviews [Bengio et al. 2021; Talbi 2021] present taxonomies that differentiate the ways and the stages in which DL can be applied to COPs: end-to-end methods, improvement methods, and hybrid methods. Nonetheless, end-to-end methods, also called **Neural Combinatorial Optimization (NCO)** methods [Bello et al. 2016], present an innovation, as they bring a different way of optimizing COPs under a learning-inference scheme, an idea not observed until recently. In this two-step approach, a model is first trained, denoted as learning phase, looking for the best parameter values. Once the model has been trained, it can be applied to any instance (inference phase), for which we aim to provide the best possible solution.

Few works have compared their performance to the existing state-of-the-art metaheuristics for the approached problem. In fact, the issues arising from the comparison of metaheuristic algorithms and NCO methods have not been addressed exhaustively. In this work, we present a critical analysis on the incorporation of NCO algorithms into the classical combinatorial optimization framework, focusing on four fundamental aspects from the optimization perspective: (1) **Performance**. How good are the solutions provided by these models? Are they competitive with the state-of-the-art methods? (2) **Computational cost**. Considering the computing resources and the time required to train these NN-based models, are they affordable for real-world problem sizes? (3) **Training data & Transferability**. Which type and amount of data is required to train the model? Is it necessary to have hundreds or even thousands of instances of the real-world problem to train the model? Can the model transfer the knowledge learned from random instances to other benchmarks and/or other sizes? (4) **Model Reusability**. Classical heuristics are carefully handcrafted by using the expert knowledge about the optimization problem. Conversely, NCO methods claim the ability to autonomously learn effective heuristics without any human interaction [Bello et al. 2016; Kwon et al. 2020]. But, how easily can these NCO architectures be applied to different problems without using problem-specific knowledge?

To illustrate the analysis, we guide the reader during the process of implementation and evaluation of an NCO model, exhaustively addressing the aforementioned aspects. To that end, and to enrich the discussion, we choose two practical cases: the **Linear Ordering Problem (LOP)**

[Ceberio et al. 2015] and the **Permutation Flowshop Scheduling Problem (PFSP)** [Gupta and Stafford Jr 2006], well-known NP-hard COPs but with few or non-existent NCO studies for each of the problems. Conducted experiments show that, if both solution quality and computing time are considered, metaheuristics and NCO proposals would belong to the Pareto front (non-dominated to each other), the former providing better solutions and the latter requiring shorter execution times. Regarding NCO methods, they are able to beat classical constructive proposals with a fast inference time, which makes them potentially interesting as a substitute for heuristics in tasks such as online optimization or hybridization purposes. Moreover, the NN model is capable of generalizing the learned knowledge to other types of instances and even to larger instances than those used for training. The experimentation illustrates that it is possible to reuse the proposed NCO model by applying some transformations to the optimization problem, but it has a certain loss of performance. Not limited to that, a number of promising research lines are described for future investigations on the application of end-to-end models over COPs.

The rest of the article is organized as follows: Section 2 introduces a review of meaningful works in the NCO framework. Section 3 presents four fundamental aspects of NCO algorithms and analyzes the current literature based on them. The analysis is illustrated with a case of study of a NCO model specifically developed for this work and applied on the Linear Ordering Problem and the Permutation Flowshop Scheduling Problem, which are defined in Section 4. A broad set of experiments is conducted in Section 5 and obtained results are discussed in Section 6, where we also suggest new directions for future work in the NCO area. Section 7 concludes the article.

2 ADVANCES ON NEURAL COMBINATORIAL OPTIMIZATION

In general terms, the optimization process of end-to-end models follows a training-inference pipeline. In the training phase, a set of instances is used to learn the parameters of the NN model. In this step, two main learning scenarios arise: **Supervised Learning (SL)** and **Reinforcement Learning (RL)**.

In SL, the NN model learns to imitate an optimal (or good) policy [Gasse et al. 2019]. The NN model is fed with a collection of labeled instances (<instance, best solution>), and this requires each instance to be previously solved by means of an exact or approximate solver. An example of this approach can be seen in Vinyals et al. [2015], where the authors propose an end-to-end algorithm to solve the Traveling Salesman Problem.[1] They used an architecture called Pointer Network, a sequence-to-sequence model that embeds the information of the instance and constructs a solution for the problem iteratively, adding a city to the solution at a time.

Using SL presents some serious drawbacks, since obtaining a large set of labels is not usually tractable and affects the applicability of the method [Bengio et al. 2021]. Moreover, using SL may fail to abstract the problem knowledge when the policy used to obtain the labels is suboptimal or there are multiple optimal solutions. In contrast, RL has proven to be a more suitable procedure for solving COPs. In this scenario, an agent learns how to act, without supervision, based on the rewards it receives through its optimization process, i.e., by experience [Sutton and Barto 2018]. In that context, Bello et al. [2016] introduced the NCO framework, which uses RL to train a NN model to approximate solutions for (a set of) combinatorial problems in an end-to-end manner. In that paper, the authors outperformed the model presented by Vinyals et al. [2015], using a similar architecture but replacing SL with RL. Furthermore, Bello et al. [2016] achieved better results than a classical heuristic [Christofides 1976] and OR-Tools' local search algorithm [Google 2016] on

[1] Among the most popular combinatorial problems, the **Traveling Salesman Problem (TSP)** has been one of the most studied problems. Given n cities and the distance between each pair of cities, the goal in TSP is to find the shortest route that passes through all the cities once.

TSP instances up to 100 cities. At this point, it is worth noting that although the work by Bello et al. [2016] is definitively remarkable, exact methods are able to solve 85,900-city instances to optimality, while problems with 100 cities can be solved in a matter of seconds [Applegate et al. 2006].

Motivated by the good results obtained by NCO in Bello et al. [2016], most of the papers dealing with end-to-end models have followed a similar learning scheme, and the scientific advances have focused on changes in the used NN model architecture. As an improvement to Pointer Networks [Bello et al. 2016; Vinyals et al. 2015], **Graph Neural Networks (GNN)** [Cappart et al. 2021a] address the limitation of having an order-invariant input, i.e., GNNs are capable of representing the features of the problem without considering any specific order of the input sequence. Khalil et al. [2017a] proposed a GNN architecture called **Structure2Vector (S2V)**, which automatically learns policies for several graph problems such as **Maximum Cut (Max-Cut), Minimum Vertex Cover (MVC)**, and TSP. They implement a greedy meta-algorithm design where solutions are constructed by appending graph nodes (items) sequentially to the solutions based on the graph structure to satisfy the constraints of the problem. Concerned by the ability of GNNs to solve very large graphs, Manchanda et al. [2019] demonstrated the capability of a **Graph Convolutional Neural Network (GCN)** to solve instances of **Maximum Coverage Problem (MCP)**, MVC, and **Influence Maximization (IM)** with up to millions of nodes. However, those sizes are difficult to solve for problems represented by dense or fully connected graphs, such as the TSP. In that sense, a comparable case is the work by Ma et al. [2019], which studied the use of a so-called **Graph Pointer Network (GPN)** that combines Ptr-Nets [Vinyals et al. 2015] and GNNs [Cappart et al. 2021a] with a Hierarchical Reinforcement Learning framework to solve TSP instances with up to 1,000 cities.

With the introduction of the Transformer architecture [Vaswani et al. 2017] and due to its good performance in multiple fields, mainly in natural language processing, recent works have tried to apply its main component, the attention mechanism, to solve different COPs. Deudon et al. [2018] and Kool et al. [2018] trained an architecture based on attention mechanisms to solve routing problems, improving previously reported results [Bello et al. 2016; Khalil et al. 2017a; Vinyals et al. 2015]. More recently, Kwon et al. [2020] achieved even better performance for the TSP, specifically, they obtained on average a gap of 0.14% with respect to the optimum value in TSP instances of 100 cities. They used the model from Kool et al. [2018] and presented an approach, called POMO, that takes different initializations of the same instance and forms a batch to perform the inference. Finally, Kwon et al. [2021] propose a double-GNN with attention mechanisms that operates on complete bipartite graphs, making it suitable for problems based on data matrices, such as scheduling problems and linear/quadratic assignment problems.

The race to propose more efficient algorithms is bringing significant progress to the NCO field. Nevertheless, some aspects, such as training cost, transferability, and reusability, are put aside in these works. In what follows, we present a broad analysis to address the concerns that arise from the introduction of NCO in the conventional optimization paradigm and propose good practice guidelines to follow.

3 CRITICAL ANALYSIS

As discussed previously, NCO models seem to be a valuable tool in the field of combinatorial optimization. But are they competitive on a standalone basis with state-of-the-art algorithms? From the viewpoint of a combinatorial optimization practitioner, carrying a performance comparison between NCO and **Conventional Optimization Algorithms (COAs)**, such as metaheuristics, presents a number of problems that need to be answered. The main difference between the application of COAs and NCO models comes from the optimization pipeline.

The general pipeline followed in a conventional optimization process usually starts with an (a set of) instance(s) to be solved and a computational budget. Depending on the problem class and the budget, an algorithm (or many) is selected along with its hyper-parameters, whose values are usually set to those recommended by the author(s). Subsequently, the algorithm starts the optimization process and once the budget expires, a result (solution to the problem instance) is provided.

Conversely, DL, and consequently NCO, runs a different pipeline. As mentioned in the introduction, the NCO pipeline involves two phases. The first, the training phase, consists of minimizing a loss function by adjusting the weights of a neural network and requires a significant amount of time and resources. Once the weights are fixed, the model is ready to be used (inference phase), providing, in a short time, a solution for each instance that is given. The inference phase can be repeated for several problem instances without the need of training the model again. This makes NCO suitable for optimization problems that need to be solved frequently and in a short period of time. Conversely, current state-of-the-art metaheuristics become superior when given a larger time budget. Placing both approaches side-by-side, it is easy to see that NCO introduces an optimization pipeline that clashes with the conventional framework in a number of aspects.

In what follows, we address the unanswered questions focusing on the presented four interrelated aspects that need to be studied to fairly compare these models with COAs and contribute to the scientific progress.

Performance analysis. When developing NCO approaches for a given problem, comparing their performance to the current state-of-the-art proposals is a must, not with the purpose of invalidating the NCO algorithm, but to put it into perspective, as is done when evaluating COAs. In that sense, making a fair comparison between NCO and COA is not trivial, as the experimental setup used in both paradigms differs. Traditionally when comparing COAs, two different stopping criteria are used: a limited computation time or a fixed number of objective-value evaluations, each of them having their supporters and detractors. In fact, most of the COAs have the ability to improve results if a larger budget is available, making it also difficult to establish a fair enough limit for all the algorithms included in a comparison. Therefore, reporting the real objective values obtained by each of the proposals, NCO and COA, in the comparison, although not using the same budgets, seems to be the only rigorous way to conduct a pure performance analysis.

Computational cost. NCO models need to be trained for several epochs, and once the training of the model has finished, they are ready to infer (solve) a large number of instances in a short period of time. Conversely, COAs face each instance individually and start the optimization procedure from scratch, without generally any knowledge transfer from instance to instance. In a real scenario, in which the model will be applied continuously to new instances, the training time could be ignored. However, the training time should be considered in the computational cost evaluation of NCO works, as it can be significant based on how often the model has to be updated (re-trained).

It should be noted that both COAs and NCO methods have specific hyper-parameters, such as population size or mutation rate for the former and the number of layers or the learning rate for the latter. As in both cases these hyper-parameters need to be tuned to optimize their performance, we decided not to include this calibration step in the computational cost analysis.

Another issue when comparing the computational efficiency of the available algorithms comes from the different programming languages used to code them and on the hardware in which they run. While COAs are generally written in *C/C++* and deployed in CPUs, NCO models are mostly implemented in *Python* and use libraries optimized to carry out parallelized training and inference on GPUs. To perform a fair comparison, we should implement them in the same programming language and try to run them on similar hardware, which is neither natural nor efficient. So, it seems reasonable to compare the algorithms implemented and executed in the programming

languages and hardware infrastructure, which the final practitioner will have easy access to, without the requirement of large overheads.

It has been broadly reported that the use of exact methods is intractable with very large NP-hard instances, as the required computation time grows exponentially. Similarly, there is a fairly high memory/computation cost when training NCO models, which grows with both the training batch and the instance size. For this reason, an analysis on the time and memory consumption of these algorithms can give an intuition of their limitations.

Training & Transferability. In the optimization field, COAs are tested on different functions and/or instances of a given problem. To measure and compare their performance, common testbeds are required such as real-world benchmarks. Unfortunately, as large sets of real-world problems (instances) are difficult to obtain, NCO models are usually trained using randomly generated instances. However, designing random generators capable of drawing instances from the desired target distribution is not trivial. Instead, a common strategy is to use uniform-distribution random generators, which are simple to create, even though they may not be a faithful reflection of real problem instances.

Besides that, it is key to study the transferability of NCO models, i.e., the capacity to apply learned insights to diverse test instances. This becomes particularly important when (1) test instances differ in size, (2) have different characteristics that are not covered by the used generator, or (3) when models are trained on small real-world sets. These situations could result in model overfitting and limited transferability.

Model reusability. As stated in the introduction, Bello et al. [2016] and Kwon et al. [2020] claim that NCO methods do not need any problem-specific knowledge. In our opinion, this statement is partially true. On the one hand, no knowledge is explicitly required during the training and inference phases. On the other hand, even if the model is applicable to a particular problem, to design a good-performing NCO model, using problem-specific knowledge during the design of its architecture is essential. Thus, studying model reusability in terms of applicability to (and performance on) different problems would help to explore new NCO architectures.

3.1 Literature Analysis

Once we have introduced the key aspects that, in our opinion, should be considered in all the new NCO proposals, let us examine the existing literature in the field, focusing on the aforementioned four aspects. Exact details have been collected in Table 1. In what follows, a summary is provided:

Performance. As can be seen in Table 1, most of the NCO models have been applied to routing problems. Regarding performance, the best proposals are able to beat constructive or baseline approaches, but they are outperformed by the state-of-the-art algorithms, which can solve large TSP instances in only a few seconds [Applegate et al. 2006]. As an exception, Manchanda et al. [2019] claim that their algorithm is marginally better than the state-of-the-art for the Influence Maximization problem.

Computational cost. In the current literature, almost no information is provided regarding the training phase, such as the GPU hours and the type of GPU used, which are crucial for estimating training costs. The context in which the NCO model will be implemented conditions the impact of the training time. For example, a user that needs to solve a COP once a day could consider using a metaheuristic or even an exact method. Conversely, in an environment where problem instances are solved in a matter of seconds, NCO approaches could have a place, the training time being negligible.

Training data & Transferability. Most of the works use randomly generated instances for training, following a common trend in combinatorial optimization. However, the influence of the instances used for training and the transferability of the models to other sets of instances has not

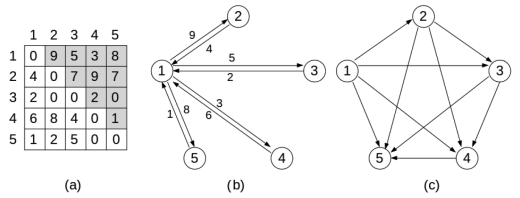


Fig. 1. Example of a LOP instance of size $n = 5$. **(a)** The LOP instance matrix of size $n = 5$ ordered as the identity permutation $\pi_e = (1\ 2\ 3\ 4\ 5)$. Entries of the matrix contributing to the objective function are highlighted in grey, the sum of the entries in the upper diagonal gives the objective value, which is 51. For this instance, the optimal solution is given by the permutation $\pi = (4\ 1\ 2\ 5\ 3)$ with an objective value of 60. **(b)** Equivalent complete graph with edge weights; only the edges of the first node are shown for clarity. **(c)** The solution graph incorporating the identity permutation $\pi_e = (1\ 2\ 3\ 4\ 5)$. The first node in π_e shows only outgoing edges, indicating its precedence over others, while the last node has only incoming edges.

been studied in depth. In addition, models are mostly applied to toy-size instances, which may be due to the limitations of the models and their lack of scalability, both in terms of computational resources and/or performance.

Model reusability. As described in Table 1, the vast majority of works tackle routing problems, which has become the main playground of NCO algorithms. As a result, models for dealing with routing problems, or those that have an underlying graph structure, have been intensively studied. However, these models have not been tested on problems of a different nature, such as assignment, cutting, or packing, and thus, model reusability remains to be an open line for research.

As can be observed, the four main aspects have not been properly studied in the NCO literature. Thus, to bridge the gaps among all those works and provide an answer for the questions that we have raised, we conduct an exhaustive evaluation and a critical analysis of a purpose-built NCO model, designed for solving two well-known permutation based problems: LOP and PFSP.

4 CASE OF STUDY

In this section, we will describe the studied problems: the Linear Ordering Problem and the Permutation Flowshop Scheduling Problem; Subsequently, we will illustrate the implementation of the model that fits their characteristics.

4.1 Linear Ordering Problem

The **Linear Ordering Problem (LOP)** [Ceberio et al. 2015] is a classical COP. Particularly, the LOP is a permutation problem that, in 1979, was proven to be NP-hard by Garey and Johnson [1979]. Since then, and due to its applicability in fields such as machine translation [Tromble and Eisner 2009], economics [Leontief 1986], corruption perception [Achatz et al. 2006], and rankings in sports or other tournaments [Anderson et al. 2021; Cameron et al. 2021], the LOP has gained popularity and it is easy to find a wide variety of works that have dealt with it [Anderson et al. 2022].

Given a matrix $B = [b_{ij}]_{n\times n}$, the goal in the LOP is to find a simultaneous permutation of rows and columns such that the sum of the upper triangle entries is maximized (see Figure 1(a)). The objective function is defined formally as in Equation (1), where π represents the permutation that simultaneously re-orders rows and columns of the original matrix and n is the problem size.

$$f(\pi) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} b_{\pi_i \pi_j} \tag{1}$$

Table 1. Analysis of the Most Relevant NCO Works in the Literature Considering the Main Optimization Aspects

Ref	Problem	Model	Learning	Performance	Time Cost	Training data & Transferability	Model Reusability
[Vinyals et al. 2015]	TSP, CH, DT	Ptr-Net	SL	Limited to the imitated algorithm. Outperforms sequence-to-sequence models.	Time cost not clearly shown	Only random instances used for training and testing. Up to TSP50.	Not size-invariant architecture. The model can be used for problems where the solution size depends on the number of elements in the input sequence.
[Bello et al. 2016]	TSP, KP	Ptr-Net	RL (PG)	Better than [Vinyals et al. 2015]. Still far from the state-of-the-art Concorde solver.	AS is expensive and not useful compared to an extensive search. While not being size-invariant, training time should be considered.	Only random instances used. TSP20, 50, 100. KP50, 100, 200.	Not a size-invariant model. Same as [Vinyals et al. 2015]. Two problems studied.
[Khalil et al. 2017a]	MVC, Max-Cut, TSP	S2V (GNN)	RL (DQN)	Compared to approximation heuristics (not s.o.t.a.) DQN based algorithms seem inferior to PG algorithms.	11 second inference for MVC graphs with 1,200 nodes	Trained with random instances, test with random and benchmarks (MemeTracker, Physics and TSPLIB). Up to TSP1, 200.	Suitable for graph-based problems.
[Manchanda et al. 2019]	MCP, MVC, IM	GCN	RL (DQN)	Marginally better than state-of-the-art for Influence Maximization.	Efficient for problems with very large graphs. Orders of magnitude faster than previous works.	Trained with generated instances, tested on real-world benchmarks. Up to millions of nodes.	Suitable for graph-based problems.
[Deudon et al. 2018]	TSP	Attn-Net	RL (PG)	Real model performance is shadowed by applying 2OPT on top of the best sampled solution.	All instances solved within a fraction of a second	Only random instances used for both training and testing. Up to TSP100.	Attn-Net can be used for various problems, but only TSP is studied.
[Kool et al. 2018]	TSP, OP, PCTSP	Attn-Net	RL (PG)	Better than previous Ptr-Net and GNN algorithms. Although having a similar architecture, outperforms [Deudon et al. 2018]. Not competitive to state-of-the-art OR solvers.	An order of magnitude faster than [Bello et al. 2016].	Only randomly generated instances are used. Up to TSP125.	Model usability studied for three different routing problems.
[Ma et al. 2019]	TSP, TSPTW	GPN	RL (HPG)	Worse than Attn-Net for small/medium instances. Better for large instances.	Slower than Attn-Net.	Trained in random instances and tested in generated instances and TSPLIB benchmark. Up to TSP1,000. Good generalization to larger sizes.	Any type of constrained TSP.
[Kwon et al. 2020]	TSP, CVRP, KP	Attn-Net	RL (POMO)	Best performance among NCO approaches.	Multiple greedy trajectory seems to be more efficient than sampling	Only random instances used. Only small instances used, TSP100, CVRP100, KP200.	Three different problems studied. Model is usable for different kinds of problems.
[Kwon et al. 2021]	ATSP, FFSP	MatNet	RL (POMO)	Better than simple heuristics and worse than LKH3 (ATSP). Limited comparison with metaheuristics for FFSP. Not including state-of-the-art.	Fast greedy inference and efficient multiple greedy trajectory as in [Kwon et al. 2020]	Only random instances used. Up to TSP100. Up to FFSP1, 000. Good generalization in FFSP.	Suitable for problems represented as complete bipartite graphs.

Acronyms are defined in Appendix A.

An alternative formalization of the LOP is to define it as a graph problem. Let $D_n = (V_n, E_n)$ denote the complete digraph of n nodes, where for every pair of nodes i and j there is a directed edge (i, j) from i to j and a directed edge (j, i) from j to i (see Figure 1(b)). A tournament T in E_n consists of a subset of edges containing, for every pair of nodes i and j, one of their directed edges. The LOP can be formulated as the problem of finding the acyclic tournament that corresponds to a linear ordering where the node ranked first is the one without incoming edges in T, the second node is the one with one incoming edge (from the node ranked first), and so on. The node ranked last is the one without outgoing edges in T (see Figure 1(c)). The objective of the graph problem is to find an acyclic tournament that gives a ranking (permutation) of the nodes that maximizes $\sum_{(i,j)\in T} c_{ij}$, where c_{ij} is the weight of the directed edge (i, j).

That is,

$$\begin{aligned} \max \quad & \sum_{(i,j)\in E_n} b_{ij}x_{ij} \\ \text{s.t.} \quad & x_{ij} + x_{ji} = 1, \quad \text{for all } i, j \in V_n, i < j \\ & x_{ij} + x_{jk} + x_{ki} \leq 2 \quad i < j, i < k, j \neq k \\ & x_{ij} \in \{0, 1\}, \quad \text{for all } i, j \in V_n \end{aligned} \tag{2}$$

4.2 Permutation Flowshop Scheduling Problem

The **Permutation Flow-Shop Scheduling Problem (PFSP)** is an NP-Hard combinatorial problem [Gupta and Stafford Jr 2006] that consists of a set J of n jobs that need to be processed in a set M of m production machines in the same order. A processing time for job i in machine j is given by p_{ij}, where P is an $n \times m$ matrix describing all the processing times. Every machine can only process one job at a time. The objective is to find a sequence of the jobs that optimizes a given criterion. In the literature, the most studied ones are the makespan, or the total completion time (C_{max}), and the **total flow time (TFT)**, or the sum of the times that each job remains in the flowshop ($\sum C$). In this article, we will consider the TFT criterion, as it is gaining interest recently, due to its importance in current dynamic production settings. The objective function can be described as:

$$f(\pi) = \sum_{i=1}^{n} c_{\pi_i m}, \tag{3}$$

where $c_{\pi_i m}$ is the completion time of job π_i in the last machine m and it can be recursively computed as

$$c_{\pi_i j} = \begin{cases} p_{\pi_i j} & if \quad i = j = 1 \\ p_{\pi_i j} + c_{\pi_{i-1} j} & if \quad i > 1, j = 1 \\ p_{\pi_i j} + c_{\pi_i j-1} & if \quad i = 1, j > 1 \\ p_{\pi_i j} + \max\{c_{\pi_{i-1} j}, c_{\pi_i j-1}\} & if \quad i > 1, j > 1 \end{cases}. \tag{4}$$

Note that, unlike the LOP, the PFSP cannot be naturally represented as a graph problem. However, taking the jobs as nodes of the graph, edges can be represented as the preference of the jobs in the scheduling order. This graph-based formulation will be exploited using the model presented below.

4.3 NCO Model

As mentioned in the introduction, we have designed an NCO model so we can run illustrative experiments that will enrich the discussion about the aspects identified in Section 3. With the aim of analyzing the behavior, advantages and disadvantages of end-to-end models, the designed NCO model will be applied in its basic form, laying aside any hybridization with other algorithms (heuristics, metaheuristics).

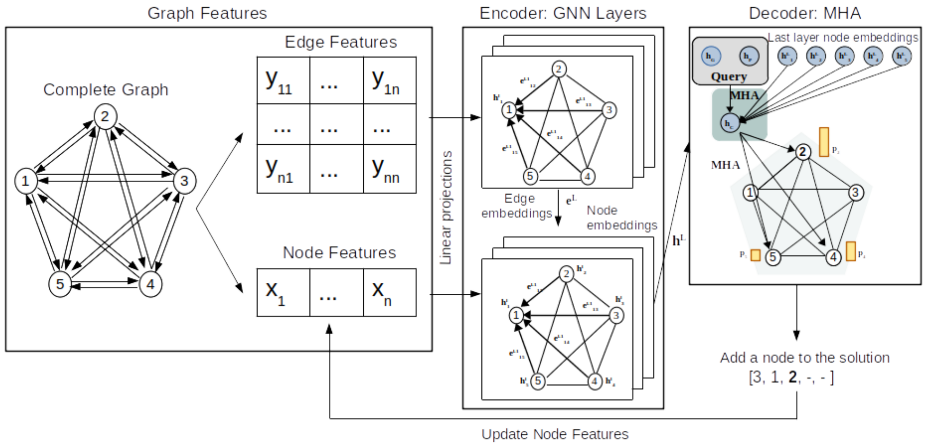


Fig. 2. The NN model architecture is composed by an encoder and a decoder. The encoder takes a learned linear projection of node and edge features and outputs a latent space representation that is transferred to the decoder. Then, the decoder uses the latent space embeddings h^L to output the vector of probabilities p for each item to be appended to the current solution.

The selection of the architecture to be used is directly conditioned by the type of problem to be solved. In our case, we aim to solve two different problems, the LOP and the PFSP. Due to efficiency reasons, the model needs to be size-invariant, so we discard sequence-to-sequence models and Pointer Networks. Instead, we opt for combining two of the most used and better performing architectures: GNNs and attention mechanism, as done in Joshi et al. [2020].

The model is autoregressive; at each step the model returns the probability of each item of the problem to be chosen (added to the solution). So, starting from an empty solution, iteratively, the model is asked for an item (the one with the highest probability) until a complete solution is obtained.

The proposed model has two main modules: a GNN encoder and an attention-based decoder. A general scheme of the architecture of the model is described in Figure 2. The encoder's function is to extract the graph-related information. It does so by using node and edge features to generate high-dimensional vectors, known as node- and edge-embeddings. These embeddings contain the essential characteristics of the graph's components in a form that can be effectively used in subsequent stages of the model. The decoder, however, is tasked with using the embeddings provided by the encoder and interpreting these to output a probability vector used to select the next item to be appended to the solution. While completing the solution, the model needs to be aware of which items have been already placed, thus, at each step, node features are updated according to the selected item in the previous iteration(s).

4.3.1 Graph Features. Graph features, that is, node- and edge-features, are used to provide information to the model about the problem to solve, together with the actual state (placed item(s) in the solution). In this article, we consider the following node- and edge- features for each one of the problems.

LOP features. For the LOP, the instance data is represented through pairwise precedences of every pair of nodes (see Figure 2—Graph Features). This can be naturally encoded into the edge features, since the value b_{ij} of the instance matrix gives the preference between nodes i and j. Therefore, edge features are equal to the instance value: $y_{ij} = b_{ij}$.

In addition to edge features, node features (x_i) can be employed as a codification of the actual state of the solution. Specifically, we use a binary codification that assigns $x_i = 1$ to those nodes already placed in the solution and $x_i = 0$ to the rest.

PFSP features. For the PFSP, the approach is similar to the LOP in terms of encoding the relationship (preference) between jobs through edge features. Here, y_{ij}, the edge feature, represents the preference of job i relative to job j in the solution. The preference is determined by the completion time when considering uniquely those jobs scheduled in the order (i, j).

Node features for the PFSP are designed to provide the model with insight into the potential benefits of adding each job to the solution. We use the unweighted index function described by Liu and Reeves [2001] as node feature, $x_i = IT_i + AT_i$ given by a combination of the **idle-time (IT)** and **artificial completion time (AT)** for each job if they were to be selected in the next step.

A linear projection of node and edge features forms node- and edge- embeddings, and these are fed to the encoder, which has L layers. In each layer, each node gathers information from the rest of the nodes via their connection edges forming node embeddings. Concurrently, edges form edge embeddings by collecting data from the nodes they connect (see Figure 2—Encoder). Then, in a second step, embeddings are inserted into the decoder, which applies an attention mechanism (explained in Section 4.3.3) to produce the probabilities of selecting each node and appending it to the solution being constructed (partial solution). Finally, the feature of the selected node is updated and the process repeats again (see Figure 2—Decoder).

4.3.2 Encoder. Based on previous references [Joshi et al. 2020], we decided to use a message passing Graph Neural Network as an encoder.[2] GNNs gather node (x_i) and edge (y_{ij}) features from the graph (previous step). Through a learned linear projection of these features, they generate d-dimensional representations known as embeddings. The linear projections are shown in Equation (5), where $A_x \in \mathbb{R}^{2\times d}$; A_y, B_x and $B_y \in \mathbb{R}^{1\times d}$ are learnable parameters, and $h_i^{l=1}$ and $e_{ij}^{l=1}$ denote the node and edge embeddings of the first layer ($l = 1$), respectively.[3]

$$\begin{aligned} h_i^{l=1} &= x_i^T * A_x + B_x \\ e_{ij}^{l=1} &= y_{ij}^T * A_y + B_y \end{aligned} \tag{5}$$

The encoding process consists of several message-passing neural network layers. The first layer takes node $h_i^{l=1}$ and edge $e_{ij}^{l=1}$ embeddings. In each layer, information of neighboring nodes is aggregated and, therefore, in a GNN of L layers, the features of neighbors that are L hops away are taken into account for each node.

The node h_i and edge e_{ij} embeddings at layer l are defined using an *anisotropic message passing* scheme as in Joshi et al. [2020]:

$$h_i^{l+1} = h_i^l + gelu\left(BN\left(W_1^l h_i^l + \sum_{j\in\mathbb{N}_i}(\sigma(e_{ij}^l)\odot W_2^l h_j^l)\right)\right), \tag{6}$$

$$e_{ij}^{l+1} = e_{ij}^l + gelu\left(BN\left(W_3^l e_{ij}^l + W_4^l h_i^l + W_5^l h_j^l\right)\right), \tag{7}$$

[2]We also tested with an attention-based encoder [Kool et al. 2018; Kwon et al. 2020], which is able to obtain better performance for instances up to N=40, but was not able to generalize to larger ones, and thus, we opted for the anisotropic architecture. See Appendix B for further details.

[3]Note that the learnable parameters are not dependent of the instance size (n), instead, the learned parameters are reused n times for the node projection and $n \times n$ times for the edge projection, making the model input-size invariant.

where W_1^l, W_2^l, W_3^l, W_4^l, and $W_5^l \in \mathbb{R}^{d\times d}$ are learnable parameters, *BN* denotes the batch normalization layer, which is used to stabilize the training process and reduce the internal covariate shift, σ is the sigmoid function, $\odot$ is the Hadamard product, and $\mathbb{N}_i$ is the neighborhood of node i. In the case of a fully connected graph, as in the LOP, the neighborhood consists of every other node in the graph. Instead of *ReLU* activation functions, we used *GELU* activation functions, which are commonly used with transformers nowadays (also a fully connected GNN) [Hendrycks and Gimpel 2016].

Node embeddings in the last layer h^L are combined to produce the general graph representation (Equation (8)). We follow a common practice, taking the mean value over the node representations.

$$h_G = \frac{1}{n}\sum_{i=1}^{n} h_i^L \tag{8}$$

4.3.3 Decoder. The decoder produces the probability values that are used to make a decision about the next item to place in the partial solution. To that end, the node embeddings of the last layer and the graph representation from Equation (8) are provided to the decoder. Those node embeddings form a context vector named Query (in Figure 2—Decoder), which is used by an attention mechanism [Kool et al. 2018] to obtain a probability distribution over the set of items.

The attention mechanism is a weighted message passing process where the message values acquired from the neighbors are weighted with the compatibility between the node query and the key of the neighbor. Each query vector (Q) is matched against a set of keys (K) using the dot product to measure the compatibility. In this case, the keys are the node embeddings of the last encoding layer. As noted in Vaswani et al. [2017], having multiple attention heads ($M = 8$ is suggested) allows nodes to receive different messages from different neighbors, and this strategy called **Multi-Head Attention mechanism (MHA)** turned out to be beneficial.

To build the mentioned context vector, or query, we concatenate the graph embeddings h_G from Equation (8) and the embeddings of the already placed nodes. This can be seen in Equation (9), where [,] denotes the concatenation operation and $h_P = \frac{1}{n_{placed}}\sum_{i\in\pi} h_i^L$ is the aggregation of the already placed node embeddings.

$$\hat{h}_t^c = W_c[h_G, h_P] \tag{9}$$

The context vector $\hat{h}_t^c$ gives additional intuition about the current state of the solution. Equation (10) shows the query (Q), Keys (K), and Values (V) used in the MHA.

$$h_t^c = \text{MHA}(Q = \hat{h}_t^c, K = \{h_1^L, ..., h_n^L\}, V = \{h_1^L, ..., h_n^L\}). \tag{10}$$

Finally, a second attention mechanism, between the refined context h_t^c and node embeddings h_i^L, produces the logits u_j^c of the non-placed nodes:

$$u_j^c = \begin{cases} C \cdot \tanh\left(\frac{(W_Q h_i^c)^T \cdot (W_K h_j^L)}{\sqrt{d}}\right) & \text{if } j \neq \pi_{t'} \quad \forall t' < t \\ -\infty & \text{otherwise,} \end{cases} \tag{11}$$

where the tanh function is used to maintain the logits within $[-C, C]$ ($C = 10$). The logits at the current step t are normalized using the Softmax function to produce the probabilities p_i used to select the next item i to place in the partial solution:

$$p_i = \frac{e^{u_i^c}}{\sum_j e^{u_j^c}}. \tag{12}$$

4.3.4 Learning. The model is trained via the REINFORCE algorithm [Williams 1992]. Given an instance s, the output of the model with weights θ is a probability distribution $p_\theta(\pi|s)$. The training is performed minimizing the loss function

$$\mathcal{L}(\theta|s) = \mathbb{E}_{p_\theta(\pi|s)}[-(R(\pi) - b(s)) \log p_\theta(\pi|s)] \tag{13}$$

by gradient descent, where $R(\pi) = f(\pi)$ is the reward function, which in the case of the LOP is equal to the objective value of the instance given a solution π and the negative value of the completion time for the PFSP. $b(s)$ is a baseline value that is used to reduce gradient variance and increase learning speed. To produce the baseline, we make use of a method called **self-critical sequence training (SCST)** [Rennie et al. 2017]. We make the model greedy, by making it take actions with maximum probability, and then use the resulting reward as the baseline. As a result, only samples from the model that outperform the greedy action are given positive reward.

5 EXPERIMENTATION

In the following, we will illustrate the experimental application of the end-to-end model described in the previous section as well as consider a number of algorithms to be compared in the LOP and the PFSP. As the goal is to address the emerging questions already discussed in Section 3, we will conduct a set of experiments to answer them.

5.1 General Setting

There is a general setting in common for all the experiments, which will be described below.

Instances. As depicted in Section 3, we distinguish two main sources of instances needed to train and evaluate the models and COAs: instance generators and benchmarks.

In the case of the LOP, the most evident way of creating a generator is to randomly sample each parameter of the matrix B from a uniform distribution between $(0, 1)$. Regarding benchmarks, the LOLIB [Reinelt 2002] is the most commonly used LOP library, which is composed of real-world instances (*IO* (31 instances), *SGB* (25), and *XLOLIB* (39)) and randomly generated instances that try to mimic real-world data (*RandB* (20), *RandA1* (25), and *RandA2* (25)). Both instance sources will be adopted for the experiments. See Appendix E for more details about the nature of these instances.

For the PFSP, the random instances, i.e., processing times of the jobs, are generated from a uniform distribution between $(0, 100)$. Moreover, instances from the Taillard benchmark [Taillard 1993] composed by 10 instances for each size, are used to further evaluate the transferability of the algorithms.

Hyper-parameters. We have experimented with differently sized models. The size mainly comes from the number of encoding layers L, the embedding size d of embedding vectors h and e, used through the model and the hidden layer size of the MLP. We have found out that increasing the number of layers and embedding size from 3 and 128, respectively, do not significantly improve the GNN performance, while increasing the computational cost. Thus, those values have been used, forming a model with 500k learnable parameters. For further details, see Appendix C.

Adhering to widely applied techniques, the model parameters are optimized with AdamW [Loshchilov and Hutter 2017] with a learning rate of 1e-4, beta values of 0.9 and 0.95, and a weight decay of 0.1. Finally, the used batch sizes are (512, 128, 64, and 32) for sizes (20, 30, 40, 50) in LOP and (20-5, 20-10, 20-20, 50-10) in PFSP, respectively.

Training. We train four different GNN models for each problem, using instances of sizes $n = 20, 30, 40$, and 50 for the LOP and instances of sizes (num. jobs - num. machines) $20 - 5$, $20 - 10$, $20 - 20$ and $50 - 5$ for the PFSP. Each model is trained until convergence using an early-stopping criterion with a patience of 20 validation epochs; 1,000 batches will be seen in each epoch.

Hardware and Software. Models are trained in two *Nvidia RTX 3090* GPUs with 24 GB of memory each. The NCO algorithms have been implemented from scratch in *Python 3.8* using the *PyTorch 1.10* package.[4] Conventional algorithms are written in *C++* and executed on a cluster of 55 nodes, each one equipped with two *Intel Xeon X5650* CPUs and 64 GB of memory.

Algorithms. Among the set of conventional algorithms, we distinguish three groups: exact methods, constructive heuristics, and metaheuristics. For each group, we selected the algorithms that compose the state-of-the-art, and the stopping criterion is based on the limits reported in the original works, such that it allows these algorithms to reach their performance.

Considering exact methods, a fast non-commercial exact solver called *SCIP* [Achterberg 2009] will be used to solve the LOP, and the **Discrete Optimization Global Search (DOGS)** framework [Libralesso 2020] for the PFSP. Exact algorithms will be run with a time limit of 12 h per instance and, in the case the optimal is not found, the best found solution will be reported. Among the constructive heuristics, the algorithm by Becker [1967] and the **Liu-Reeves heuristic** [Liu and Reeves 2001] **(LRnm)** are the best performing for the LOP and the PFSP with TFT criterion, respectively. As they are deterministic constructive algorithms, they will be run once until a solution is given. The implementation of the constructive heuristics and exact solvers has been made following the respective handbooks. Also, we consider state-of-the-art metaheuristics for both problems: a **Memetic Algorithm (MA)** [Lugo et al. 2021] for the LOP and a **Differential Evolution (DE)** algorithm [Santucci et al. 2015] for the PFSP. Metaheuristics will be stopped once $1{,}000n^2$ objective function evaluations are computed, a common stopping criterion for comparing different metaheuristics [Lugo et al. 2021; Santucci et al. 2015]. Regarding the NCO model described in Section 4.3, we will include the results given by a unique model (named as GNN), and the best results given by an ensemble of five separately trained models (Pop-GNN) as an additional straightforward way to exploit NCO models.

5.2 Performance Analysis

This first experiment has been designed to measure the performance of the end-to-end model compared to the algorithms described in the previous section. For this first purpose, a test-set of 1,000 random instances will be created for each problem, with instances similar (in terms of size and generator) to those used for training.

Results of the algorithms for the LOP are depicted in Table 2. The exact algorithm is capable of finding the optimum solution for values of n up to 40. However, for larger instances, it is unable to identify the optimum solution in the given time limit. Therefore, we report the gaps of the best solution found in these cases (marked with * in the table). The MA is the most competitive, in fact, it provides the best results for all sizes among the studied algorithms. The GNN model outputs good-quality solutions, with a gap between 0.2% and 0.5%, and systematically outperforms Becker's constructive. Moreover, when an ensemble of models is used (GNN-Pop) its performance increases (0.1%–0.3%) but at the cost of a larger computational effort.

Similarly, the performance results of the algorithms to solve the PFSP are depicted in Table 3. Again, the metaheuristic (DE) is the best performer, while the GNN is superior to the LRnm heuristic. While for the LOP the GNN was able to outperform the exact algorithm for $N = 50$, in the case of the PFSP, for 50 jobs and 10 machines, the exact is superior to both the constructive and GNN. Overall, the performance of the GNN is relatively worse for the PFSP compared to the LOP. This is, in our opinion, due to the different nature of the PFSP. Its fitness function, as opposed to the LOP, is harder to represent by pairwise interactions, and thus, it is not naturally learned by the GNN.

[4]The source code is available at https://github.com/TheLeprechaun25/Applicability-NCO.

Table 2. LOP. Analysis of the Performance Using Instance Sizes the Model Has Been Trained with

Method	n = 20	n = 30	n = 40	n = 50
Exact (SCIP)	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%	1.11 ± 0.50%*
MA	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%
Becker	3.38 ± 0.00%	3.44 ± 0.00%	3.35 ± 0.00%	3.27 ± 0.00%
GNN	0.24 ± 0.00%	0.29 ± 0.00%	0.41 ± 0.01%	0.48 ± 0.01%
GNN-Pop	0.14 ± 0.00%	0.18 ± 0.00%	0.28 ± 0.00%	0.34 ± 0.00%

The given value is the average and standard deviation gap (%) to the best known value for 1,000 instances over 5 different executions. Lower is better. Non-optimal results from the exact method are marked with *.

Table 3. PFSP. Analysis of the Performance Using the Cost Given by the Total Flow-time Criterion

Method	20/5	20/10	20/20	50/10
Exact (DOGS)	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%	0.95 ± 0.25%*
DE	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%	**0.00** ± 0.00%
LRnm	6.91 ± 0.00%	6.53 ± 0.00%	4.60 ± 0.00%	7.56 ± 0.00%
GNN	4.36 ± 0.17%	4.47 ± 0.41%	3.74 ± 0.20%	6.24 ± 0.24%
GNN-Pop	3.06 ± 0.00%	2.81 ± 0.00%	2.78 ± 0.00%	4.65± 0.00%

The given value is the average and standard deviation gap (%) to the best known value for 1,000 instances over 5 different executions. Lower is better. Non-optimal results from the exact method are marked with *.

Table 4. LOP. Execution Times

Method	n = 20	n = 30	n = 40	n = 50	n = 100	n = 200	n = 1000
Exact (SCIP)	0.52s	13.4s	5.3m	max	max	max	max
MA	0.10s	0.18s	0.29s	0.43s	2.5s	19.6s	20.2m
Becker	0.001s	0.002s	0.004s	0.006s	0.02s	0.10s	3.40s
GNN	0.07s	0.11s	0.16s	0.19s	0.36s	0.74s	1.1m
GNN-training	20h	41h	73h	94h	–	–	–

The term max denotes that at least one of the executions of the exact algorithm has reached the maximum time (h, m, s refer to hours, minutes and seconds, respectively).

Table 5. PFSP. Execution Times

Method	20/5	20/10	20/20	50/10	100/10	200/20	500/20
Exact (DOGS)	6m	7m	8m	max	max	max	max
DE	4m	5m	6m	7m	9m	39m	2h
LRnm	0.02s	0.04s	0.08s	0.5s	3s	24s	4m
GNN	0.08s	0.10s	0.13s	0.6s	4s	56s	13m
GNN-training	19h	25h	36h	97h	–	–	–

The term max denotes that at least one of the executions of the exact algorithm has reached the maximum time (h, m, s refer to hours, minutes and seconds, respectively).

5.3 Computational Cost

Together with the performance, practitioners must take into account the execution time required by an algorithm to obtain the solution, which is crucial in optimization contexts in which time restrictions are present.

Following with the setting from the previous section, Table 4 shows the execution times for the different algorithms to solve a unique instance of the LOP. With the increase in size, the exact algorithm quickly reaches non-affordable execution times, while the constructive (Becker) scales successfully. The time required by the metaheuristics grows quadratically on n, which can be a bottleneck for larger sizes, but for the sizes tested in this work the execution time is still reasonable. Once trained, the GNN shows a very fast response time, just a few minutes for the largest size tested.[5]

The experiments for the PFSP show a similar trend. As can be seen in Table 5, the constructive (LRnm) is the fastest algorithm. However, the GNN requires a slightly larger budget compared to LRnm, even though it is capable of performing the inference of the largest studied sizes (500/20) in only a few minutes. Finally, the exact method is again the most time-costly, while the metaheuristic (DE) is the second. Compared to the GNN, DE is 30 times slower for instances of 500 jobs and 20 machines. For a complete overview of the Pareto analysis of different algorithms considering both solution quality and computational cost, see Appendix D

Apart form the inference time, one should also consider the training time of GNNs, which is the most time-demanding step, taking several hours; up to 29 hours for LOP instances of size $n = 50$, and is unaffordable for sizes larger than 50 with our hardware. In our opinion, training time is a relevant aspect in scenarios that require the model to be updated continuously, but could be ignored if this is not the case.

To complete the experiment, an analysis of the memory consumption has been conducted. Constructive algorithms, as well as metaheuristics, follow a linear growth, which makes the memory requirements affordable. However, the memory consumption of the training of GNNs grows poly-

[5]While the training is performed in batches, the model infers just one instance at a time. However, note that it could infer a batch of instances instead, reducing the total computation time due to parallelization.

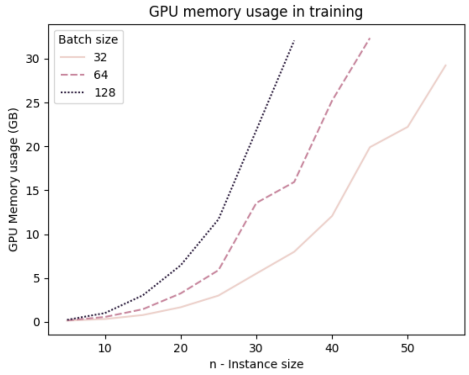


Fig. 3. GPU memory usage for different instance and batch sizes during training.

Table 6. LOP. Performance of GNN Models in Different Instance Sizes

Method	n = 20	n = 30	n = 40	n = 50	n = 100	n = 200	n = 400	n = 700	n = 1,000
GNN-20	**0.24** ± 0.00%	0.32 ± 0.01%	0.39 ± 0.01%	0.46 ± 0.02%	0.59 ± 0.03%	0.62 ± 0.04%	0.62 ± 0.05%	0.62 ± 0.08%	0.62 ± 0.14%
GNN-30	0.24 ± 0.01%	**0.29** ± 0.01%	**0.34** ± 0.01%	**0.40** ± 0.01%	0.51 ± 0.03%	0.56 ± 0.03%	0.59 ± 0.05%	0.58 ± 0.05%	0.53 ± 0.05%
GNN-40	0.32 ± 0.01%	0.38 ± 0.01%	0.41 ± 0.01%	0.45 ± 0.01%	**0.49** ± 0.01%	**0.46** ± 0.01%	**0.43** ± 0.01%	**0.39** ± 0.01%	**0.34** ± 0.01%
GNN-50	0.35 ± 0.01%	0.41 ± 0.01%	0.44 ± 0.01%	0.48 ± 0.01%	0.51 ± 0.02%	0.48 ± 0.03%	0.45 ± 0.03%	0.40 ± 0.02%	0.35 ± 0.02%

The given value is the average and standard deviation gap (%) to the best known value for 1,000 instances over 5 different executions. GNN-XX refers to the model trained with instances of size n = XX. Best average gap values are highlighted in bold.

nomially with respect to the model and the batch size. Figure 3 shows, for the LOP, the memory usage curves according to the batch size, where different curves are plotted for regularly used batch sizes. It can be seen that the training phase of GNN models is really memory-intensive, which quickly limits their applicability.

5.4 Transferability to Different Instance Sizes

Considering the huge computational resources required by NN-based models to solve large size instances, size-invariant models are really valuable. That is, models that have been trained using small instances and are later applied to solve larger ones. However, they may lack the ability to generalize the learned knowledge into larger and more complex instances, not guaranteeing competitive performances [Joshi et al. 2020].

That being said, we conduct experiments to investigate the generalization ability of GNN models to different instance sizes. For the LOP, we report results provided by models trained with instances of $n = 20, 30, 40$, and 50, and evaluated in instances up to $n = 1{,}000$. In view of the results (see Table 6), the GNN model shows a good generalization capability, as the difference with the best solution worsens slightly (gaps from 0.5% to 0.6%). Regarding generalization, more complex models, such as GNN-40 or GNN-50 show, in general, a higher performance than the more simple ones (GNN-20 and GNN-30), which is the behavior one would expect. In addition to the performance, it is important to remember again the quick response time of these models, an aspect that can be relevant in some scenarios.

Additionally, we illustrate the performance of GNNs with the rest of the proposals as a function of the instance size. Figure 4 shows how the performance gap between the best performing algorithm (MA) and the GNN model increases from n = 20 to n = 200, remains constant from n = 200 to n = 400 and slightly decreases for larger sizes, which values the generalization capacity of the

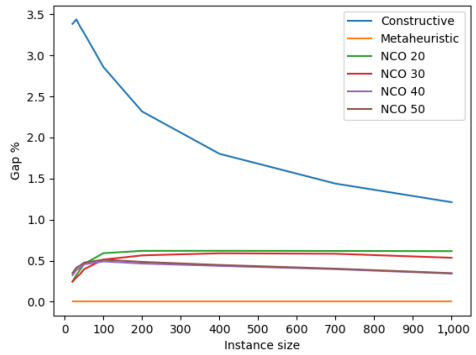


Fig. 4. LOP. Gap (%) to the best known objective value as a function of the size of instances for the analyzed algorithms.

Table 7. PFSP. Performance of GNN Models in Different Instance Sizes

Method	20/5	20/10	20/20	50/10	100/10	200/20	500/20
GNN-20-5	**4.36** ± 0.17%	4.60 ± 0.26%	4.03 ± 0.21%	6.24 ± 0.78%	7.07 ± 1.00%	7.53 ± 1.31%	7.29 ± 1.50%
GNN-20-10	5.67 ± 0.40%	**4.47** ± 0.41%	3.80 ± 0.34%	6.37 ± 0.75%	7.23 ± 0.89%	4.99 ± 0.81%	4.14 ± 0.77%
GNN-20-20	10.03 ± 0.97%	5.54 ± 0.37%	**3.74** ± 0.20%	8.99 ± 0.44%	9.30 ± 0.53%	5.12 ± 0.67%	4.12 ± 0.62%
GNN-50-10	7.10 ± 0.33%	5.16 ± 0.16%	4.03 ± 0.16%	**6.24** ± 0.24%	**6.26** ± 0.34%	**4.00** ± 0.62%	**2.85** ± 0.43%

The given value is the average and standard deviation gap (%) to the best known value for 1,000 instances over 5 different executions. GNN-XX-YY refers to the model trained with instances of n = XX jobs and m = YY machines. Best average gap values are highlighted in bold.

GNN models. Regarding the constructive algorithm (Becker), it reduces its gap with all the other algorithms consistently, even though the improvement decelerates for very large instances.[6]

In the case of PFSP, the experiment considers three models trained with instances of size (number of jobs/number of machines) 20/5, 20/20, and 50/10. As can be seen in Table 7, the model trained with smaller instances is the best performer for the 20/5 instances, while those trained with larger instances (20/20 and 50/10) are the ones that generalize better, even though they suffer with smaller instances. However, in PFSP one not only needs to consider the size of the permutation (the number of jobs) but also the number of machines. In fact, it has a considerable effect on the performance of the model; note that GNN20-20 outperforms GNN50-10 in instances of 200 jobs and 20 machines. We can conclude that: (1) the closer the number of jobs and machines to the instance used for training to that of the inference, the better the performance; (2) for large instances, which are usually intractable to solve with exact methods, the practitioner should consider training the model with the largest instances possible, always within the budget limits.

5.5 Training Data & Transferability to Target Benchmarks

To analyze the transferability of the learned model to other type of instances, and given that our purpose is to solve a certain target set of instances, we will consider three different training setups:

(1) The model is trained with random instances obtained from generators, and then target instances are solved using the model.
(2) The model is trained in two steps, first, with random instances obtained from generators, and then an additional training is performed using a small set of instances sampled from the target set of instances. In this setup, 100 epochs will be dedicated to both training steps.

[6]Although not shown in the figure, Becker obtains a gap of 0.92% for n = 2,000.

Table 8. LOP. Analysis of the Performance in Six Types of Instances (with Different Instance Sizes in Brackets) from the LOLIB Benchmark

Method	IO (44)	RandB (50)	SGB (75)	RandA1 (150)	RandA2 (200)	XLOLIB (250)
MA	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Becker	7.14%	7.49%	4.17%	6.59%	1.53%	7.73%
(1) GNN-Random	1.93 ± 0.33%	1.23 ± 0.15%	4.97 ± 0.97%	1.37 ± 0.08%	0.51 ± 0.07%	4.91 ± 0.73%
(2) GNN-Int	0.28 ± 0.08%	0.92 ± 0.10%	2.88 ± 1.06%	1.30 ± 0.07%	0.29 ± 0.01%	**4.00** ± 0.47%
(3) GNN-IO	**0.27** ± 0.02%	1.71 ± 0.16%	9.46 ± 6.07%	2.60 ± 0.13%	0.66 ± 0.04%	10.20 ± 1.48%
(3) GNN-RandB	2.35 ± 0.47%	0.81 ± 0.15%	8.68 ± 2.59%	1.42 ± 0.12%	0.54 ± 0.12%	5.73 ± 1.27%
(3) GNN-SGB	0.52 ± 0.36%	2.11 ± 0.44%	**2.63** ± 1.48%	2.62 ± 0.45%	1.16 ± 0.40%	5.62 ± 0.47%
(3) GNN-RandA1	2.23 ± 0.43%	**0.69** ± 0.18%	7.69 ± 1.54%	**1.09** ± 0.06%	0.45 ± 0.02%	4.36 ± 1.05%
(3) GNN-RandA2	5.59 ± 3.62%	0.99 ± 0.12%	17.22 ± 8.83%	1.51 ± 0.08%	**0.26** ± 0.03%	9.89 ± 2.61%
(3) GNN-XLOLIB	0.36 ± 0.13%	2.69 ± 0.32%	3.05 ± 2.08%	2.98 ± 0.11%	0.94 ± 0.13%	5.96 ± 0.29%

GNN-Random is the model trained with random instances of $N = 40$. The average gap (%) and standard deviation (%) are computed with respect to the best known value given by the Memetic Algorithm [Lugo et al. 2021].

(3) The model is trained using a subset of instances sampled from the target set without any previous training.

For the first and the second cases, we will use uniformly distributed instances, and instances from the LOLIB and Taillard benchmarks will constitute the target set for the second and third cases. For the experiment, half of the benchmark instances will be used to form the training set and the other half will be used for testing.

Table 8 gathers the results obtained for the different setups in LOP. Even if LOLIB instances are heterogeneous, regarding their origin or the procedure used to create them, training the model with random instances (setups 1 and 2) is notably better than training the model directly using the set of instances we want to solve. However, it can be observed that GNNs trained with random instance generators perform generally better in random instances (*RandA1*, *RandA2*, *RandB,* and *MB*) than in real-world instances (*IO*, *SGB,* and *XLOLIB*). So, the use of generators is advisable, but these generators should be able to produce instances as close as possible to the ones we want to solve.

Finally, as seen previously, including an additional training phase with instances from the target distribution is really helpful (setup 2—GNN-Int), providing the best performing models (gaps between 0.5% and 3.72% with respect to the best solutions found).

This experiment has also been designed to give an intuition about the transferability between different LOLIB benchmarks. That is, considering the computation effort required to train a model, can this model be applied successfully to other types of instances? In this regard, interesting results have been found. Remarkably, training in *SGB* is transferable to *IO*, training in *XLOLIB* is transferable to *SGB*, and training in random instances is transferable to the rest of random benchmarks.

Regarding the PFSP, Table 9 shows that the intensification phase does not achieve such good results. Instead, models trained with a larger set of randomly generated instances yield better performance. Overall, the gap between the LRnm heuristic and GNNs is smaller in Taillard instances, even though, in general, the GNN is still better.

5.6 Model Reusability

In addition to the aspects analyzed throughout this section, model reusability is also a desired characteristic for optimization algorithms. That is, a model that is general enough to be used in many different problems without much need for tuning. The GNN model designed in this article fulfills this property, as it can be applied "as it is" to any graph-based optimization problem. Of course, the practitioner must feed the network with the appropriate node and edge feature values, and the performance may not be good enough. In fact, even though both problems can be approached

Table 9. PFSP. Transferability Analysis with Instances from the Taillard Benchmark

Method	Tai20/5	Tai20/10	Tai20/20	Tai50/10	Tai100/10	Tai200/20	Tai500/20
DE	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
LRnm	10.87%	6.76%	**1.95**%	7.12%	**7.24**%	7.86%	4.29%
(1) GNN-20-5	**5.21** ± 0.76%	4.17 ± 0.87%	5.11 ± 0.33%	8.37 ± 1.02%	9.88 ± 1.26%	9.49 ± 2.23%	7.41 ± 3.27%
(1) GNN-20-10	6.70 ± 1.32%	**3.68** ± 0.33%	4.97 ± 0.63%	**5.81** ± 0.89%	9.47 ± 1.18%	7.97 ± 0.52%	4.60 ± 0.88%
(1) GNN-20-20	11.29 ± 1.44%	5.27 ± 1.57%	4.42 ± 0.38%	7.14 ± 0.80%	12.12 ± 1.28%	7.50 ± 0.37%	4.82 ± 0.57%
(1) GNN-50-10	8.97 ± 0.93%	6.42 ± 0.33%	4.69 ± 0.70%	6.46 ± 0.54%	7.90 ± 0.50%	**5.95** ± 0.54%	**2.66** ± 0.35%
(2) GNN-Int	7.01 ± 2.05%	3.77 ± 0.82%	4.09 ± 0.48%	7.40 ± 0.25%	9.49 ± 0.74%	8.83 ± 0.66%	7.01 ± 0.69%
(3) GNN-Tai	11.47 ± 1.86%	6.42 ± 1.64%	4.64 ± 1.11%	11.11 ± 3.12%	13.10 ± 3.09%	10.70 ± 3.11%	9.99 ± 4.72%

(3) GNN-Tai denotes the model trained with instances sampled from the Taillard benchmark. The average gap (%) and standard deviation (%) are computed with respect to the best known value given by the Differential Evolution Algorithm [Santucci et al. 2015].

using the proposed model, it seems that the LOP fits better with the model's characteristics, being possibly the reason why it performs better in the LOP. These results confirm that model reusability is an aspect that deserves more attention from the NCO community.

6 DISCUSSION AND FUTURE WORK

Through the experimentation section, we have tested different aspects and properties of the end-to-end model to analyze its behavior and competitiveness.

First, our observations indicate that NCO models have the potential to be general-purpose algorithms. Proof of that is the successful application of the same GNN architecture for both LOP and PFSP in this article. As the NCO field is still in early development, its effectiveness is likely to improve with anticipated advancements in the DL-optimization sector. This will also be enhanced by a deeper understanding on how to incorporate problem-specific knowledge into NCO models.

We made an effort to propose a good end-to-end model for optimizing the LOP and the PFSP, trying to find the most competitive training strategies. Although the conducted experiments showed that the NCO model obtains a remarkable performance when compared to the constructive heuristics, it is not still able to beat state-of-the-art methods (such as MA or DE). Furthermore, NN-based models have a serious drawback regarding the training time and the memory requirements as the instances become larger.

In light of the experimental results, one might wonder about the comparative advantages of using NCO models over metaheuristics. While specialized metaheuristics have produced better quality solutions across different problems, they require several hours to properly converge for the largest studied sizes. Conversely, NCO methods are able to provide a solution in a few minutes, being those solutions better than those provided by the classical constructive methods included in the study. Thus, in environments where a fast response is required (assuming some performance loss), these models are an interesting option, for example, in online decision-making optimization problems (e.g., logistics). However, it must be also noted that the training is computationally very expensive so, to be efficient, the model should not need to be re-trained frequently, as it would lose its competitiveness (fast response). In this regard, it is also worth noting the valuable advantage of input-size invariant models, such as the one designed in this work. They can be trained for n-sized instances and later be applied to $m >> n$-sized instances, maintaining a constant gap or even reducing it with respect to the state-of-the-art metaheuristics, which usually suffer, to a greater extent, with the increase of instance size.

Regarding the training process, there is another aspect that must be considered. Do we always need a large set of instances of the problem at hand to train a good-performing model? As observed in the experiments, the effectiveness of the model is greatly influenced by the quantity and the diversity of the instances used for training. While thousands of instances are required for training,

real benchmarks do not generally have enough instances. Here, random instance generators come into play. However, implementing random instance generators that produce samples with characteristics similar to the target scenarios is usually challenging. Nevertheless, we have shown that, when ad hoc generators are not available, a successful alternative is to employ uniform random generators to train a baseline model, and, if possible, incorporate an additional training phase with a small subset of real instances.

It is important to acknowledge the limitations of the experiments conducted in this study, which are constrained by factors such as the scope of the problems evaluated and the specific configurations of the NCO models tested. These limitations must be considered when interpreting the results. Considering this, experiments conducted in this work suggest that NCO methods have certain capabilities that make them an interesting case of study. Although NCO is not able to outperform state-of-the-art metaheuristics as a standalone method, its fast inference time and generalization ability can be helpful as a combination with other methods. With this purpose in mind, recent reviews [Bengio et al. 2021; Talbi 2021] identify different works that employ Machine Learning models to improve Branch and Bound algorithms, for example, deciding in which node to apply a given heuristic [Khalil et al. 2017b] or generating a high-quality joint variable assignment [Nair et al. 2020].

In our opinion, there exists another relevant research topic that falls in line with the NCO model proposed in this article. The output of the NCO model developed for this work is a probability vector that is used to guide the construction of a solution, i.e., deciding the item to place in the next empty position of the permutation. However, the NCO model could also be used to guide more advanced algorithms, such as trajectory- or population-based metaheuristics. A relevant research line consists in developing models capable of encoding the whole population of solutions as an internal state in a population-based metaheuristic; or models that encode the current solution in trajectory-based algorithms, such as **local search (LS)**. In fact, LS schemes usually employ quadratic size neighborhood structures that are computationally intensive to process. In such cases, given a problem instance and a solution, it would be very valuable to have a model that is able to propose the most promising neighborhood operation (or operations) to choose. Note that this is challenging, since, in addition to the instance, the models must codify the solution (received as input) at which the LS algorithm is at each step. Although few, there are some works that can inspire research in this direction [Chen and Tian 2019; Garmendia et al. 2022; Wu et al. 2021]. Another interesting research line involves a detailed investigation of the training dataset's composition necessary for effectively training the model. This includes designing generators that consider the problem structure and the characteristics of the target instances. In this context, the application of generative models, such as Generative Adversarial Networks [Goodfellow et al. 2020] and Diffusion models [Ho et al. 2020], appears particularly promising to automatically generate instances close to the target distribution.

Finally, it is worth noting that almost all the proposals in the literature focus on solving scheduling and routing-like problems. However, in the optimization area, we can find problems with constraints, multiple objectives, dynamic nature, and so on, and the design of NCO models to solve this kind of problem is still at an early stage. For example, when it comes to dealing with hard constrained problems, three main approaches are discussed in the current NCO literature: (1) Manually forcing the solution to fit the given constraints, restricting unfeasible actions [Cappart et al. 2021b]; (2) Adding a penalty value to the reward function for unfeasible outputs, as mentioned by Bello et al. [2016]; (3) Allowing the model to initially produce unfeasible solutions that are subsequently adjusted using a repair operator to be feasible [Zhang and Dietterich 2000]. However, instead of finding external mechanisms as the above, exploring models able to internally deal with hard constraints (but also multiple objectives, dynamism, etc.) is a key research line.

7 CONCLUSION

In this article, we conducted a critical analysis of Neural Combinatorial Optimization algorithms and their incorporation into the conventional optimization framework. The analysis consists of four interrelated axes: the performance of the algorithm, the computational cost, the transferability to different benchmarks and sizes, and the reusability of the model. In addition, we discussed the guidelines to facilitate the comparison of NCO approaches together with COAs in a rigorous manner. The conducted analysis has been enriched with the design and validation of a new learning-based algorithm composed of a graph neural network and an attention mechanism. We selected two NP-Hard problems, the **Linear Ordering Problem (LOP)** and the **Permutation Flowshop Scheduling Problem (PFSP)**, and guided the reader during the process of implementation and evaluation of the proposed model. We compared the NCO method with a diverse set of algorithms, including exact solvers [Achterberg 2009], classical constructive heuristics [Becker 1967; Liu and Reeves 2001], and state-of-the-art metaheuristics [Lugo et al. 2021; Santucci et al. 2015]. Finally, we discussed the results, pointing out future research lines in the field of end-to-end models, which can be a promising paradigm towards the design of more efficient optimization methods.

APPENDICES

A LIST OF ACRONYMS

(Alphabetically ordered)

AS - Active Search
Attn-Net - Attention Network
ATSP - Asymmetric Traveling Salesman Problem
CH - Convex Hull
DQN - Deep Q-Learning
DT - Delaunay Triangulation
FFSP - Flexible Flowshop Scheduling Problem
GCN - Graph Convolutional Network
GNN - Graph Neural Network
GPN - Graph Pointer Network
HPG - Hierarchical Policy Gradient
IM - Influence Maximization
KP - Knapsack Problem
MatNet - Matrix Encoding Network
MCP - Maximum Coverage Problem
MVC - Minimum Vertex Cover
OP - Orienteering Problem
Ptr-Net - Pointer Network
PG - Policy Gradient
PCTSP - Prize Collecting Traveling Salesman Problem
RL - Reinforcement Learning
S2V - Structure to vector
TSP - Traveling Salesman Problem
TSPTW - Traveling Salesman Problem with Time Windows

B MODEL ARCHITECTURE

We have tested two of the most used GNN architectures, the anisotropic message passing GNN [Joshi et al. 2020] and the **attention-based GNN (GAT)** [Kool et al. 2018]. We have found out that GAT obtained better performance in small instances, but it was not able to generalize to larger ones, and thus, we opted for the original architecture (see Table 10).

C MODEL SIZE

The size of the model mainly comes from the number of encoding layers and the embedding size. As can be seen in Figure 5, the optimum number of layers is 3, since it performs better than the model with 2 layers and similar to the model of 4 layers while being faster. Regarding the embedding size, Figure 6 shows that those models with a larger embedding dimension need less episodes to reach higher scores. Moreover, small dimensions (16, 32, and 64) do not reach the performance of larger

Table 10. LOP Performance Comparison of Two Architectures: Anisotropic Message Passing (GNN) and Graph Attention Network (GAT)

Method	n = 20	n = 30	n = 40	n = 50	n = 100	n = 200	n = 400	n = 700	n = 1,000
GNN	0.24 ± 0.00%	0.29 ± 0.01%	0.41 ± 0.01%	0.48 ± 0.01%	0.51 ± 0.02%	**0.48** ± 0.03%	**0.45** ± 0.03%	**0.40** ± 0.02%	**0.35** ± 0.02%
GAT	**0.08** ± 0.00%	**0.14** ± 0.00%	**0.21** ± 0.01%	**0.28** ± 0.01%	**0.36** ± 0.03%	0.78 ± 0.31%	1.73 ± 0.40%	2.17 ± 0.39%	1.98 ± 0.47%

The given value is the average gap % to the best known value for 1,000 instances over 5 different executions. Lower is better. For n from 20 to 50, the model trained with each instance is used, and for larger sizes, the model trained with $n = 50$ is used.

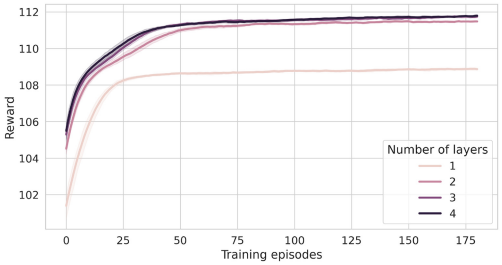


Fig. 5. Training reward curves for models with different number of encoding layers. Each line is the average among 10 different training executions. In this experiment, LOP instances of size $n = 20$ were used.

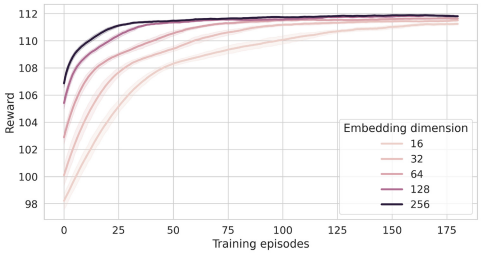


Fig. 6. Training reward curves for models with different embedding dimension. Each line is the average among 10 different training executions. In this experiment, LOP instances of size $n = 20$ were used.

dimensions. Note that increasing the embedding dimension from 128 to 256, the duration in time of an episode increases 50%, while there is almost no difference when comparing it with smaller dimensions.

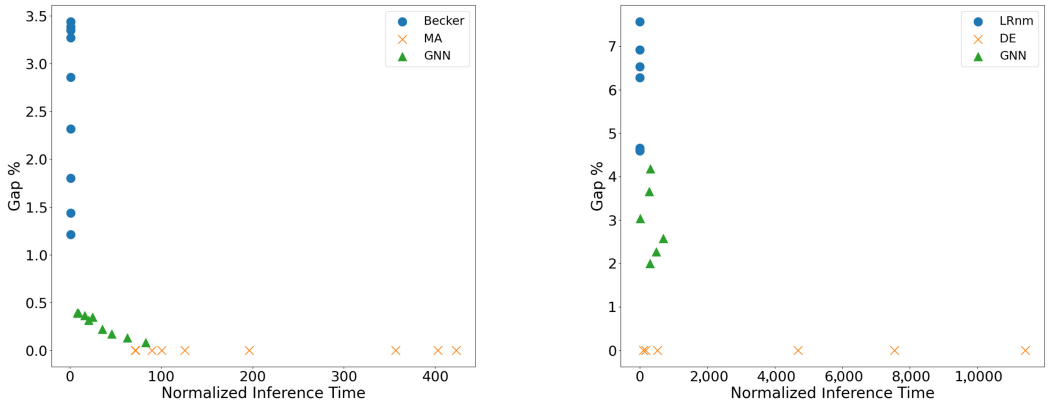
D PURE PERFORMANCE VS. COMPUTATIONAL COST ANALYSIS

To analyze the relation between two of the main aspects of the presented algorithms, we include in this section a conjoint analysis of the pure performance and the computational cost. In fact, we consider two different computational costs: the inference cost and the training cost.

Figure 7 contrasts different algorithms by examining both the quality of their solutions and the cost of the inference step. As mentioned in the results, NCO methods are an interesting choice, as they have faster inference time compared to that by metaheuristics,[7] and present better solutions than classical constructive methods.

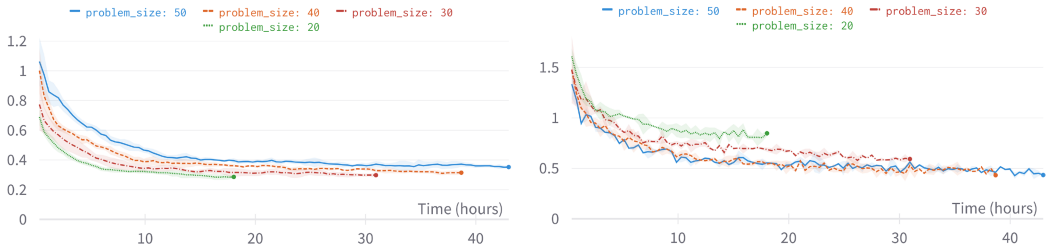
Regarding the training cost, as expected, the larger the model, the higher its time and memory requirements. Figures 8 and 9 show the evaluation gap of four GNNs of different sizes at different steps (times) of the training process. It allows us to observe the benefit that extending the training time has, together with the transferability of the models when applied to instances of larger sizes. In summary, models trained with instances that are closer to the evaluation size perform the best (see Figure 8 for illustration). Moreover, the performance increases as additional training time is allowed, being this increase more significant in the earlier steps. As concluded in Section 5, the GNN models show a better (more coherent) behavior for the LOP problem than for the PFSP. In the latter, Figure 9, pairwise interactions are harder to represent, and the GNN models with different number of machines struggle to learn a useful construction policy (poor transferability). With no doubt, additional research is needed to understand some of the erratic curves observed.

[7]Metaheuristics do not work on a training-inference pipeline, thus, in the comparison of inference times, we refer to the time needed by metaheuristics to produce a result, i.e., the time cost of the optimization process.



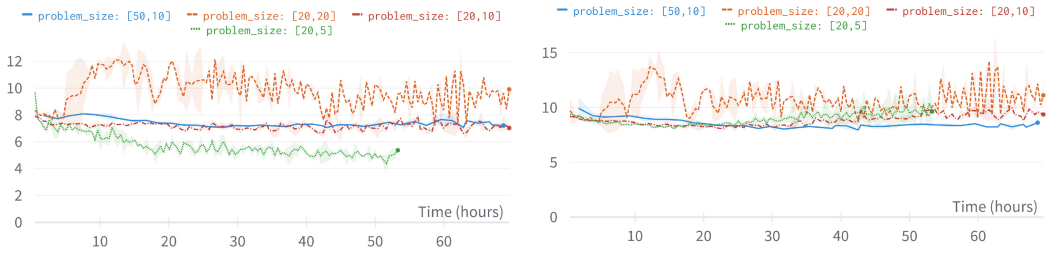
(a) LOP (b) PFSP

Fig. 7. Comparison of gap percentages against normalized inference times (with respect to the time needed by the constructive) for different algorithms in the studied instance sizes. Each point represents the average value for each instance size. GNN-40 and GNN-50-10 models have been used for LOP and PFSP, respectively.



(a) N = 30 (b) N = 200

Fig. 8. Performance (gap percentage, y-axis) for the LOP evaluated using several training checkpoints (training time in x-axis). We show the performance of GNN models trained with different instance sizes (20, 30, 40, and 50) and evaluated in instances of size 30 (a) and 200 (b).



(a) 100 jobs 10 machines (b) 20 jobs 5 machines

Fig. 9. Performance (gap percentage, y-axis) for the PFSP evaluated using several training checkpoints (training time in x-axis). We show the performance of GNN models trained with different instance sizes (20-5, 20-10, 20-20 and 50-10) and evaluated in instances of size 20-5 (a) and 100-10 (b).

E TESTING BENCHMARKS

In this section, we describe precisely the instances from the LOLIB benchmark [Reinelt 2002] and the Taillard benchmark used to evaluate the different algorithms in the LOP and the PFSP, respectively.

The LOLIB is composed by the following instance types:

- **Input-Output matrices (IO)** [Grötschel et al. 1983]. Real-world economical tables, scaled to integer values.
- **SGB** instances [Laguna et al. 1999]. Instances taken from the Stanford GraphBase. They are random instances with entries drawn uniformly distributed from [0, 25,000].
- Random instances of type **RandA1** [Campos et al. 2001]. Instances generated from a [0, 100] uniform distribution.
- Random instances of type **RandA2** [Campos et al. 2001]. Instances generated by counting the number of times an item appears in a higher position than another in a set of randomly generated permutations.
- Random instances of type **RandB** [Campos et al. 2001]. For these instances, the super-diagonal entries are drawn uniformly distributed from $[0, U_1]$ and the sub-diagonal entries from $[0, U_2]$, where $U_1 \geq U_2$. In our instances, $U_1 = 100$ and $U_2 = 100 + 2(i - 1)$, where i is the index of the instance.
- Instances from the **eXtended LOLIB (XLOLIB)**. A set of larger, random or real-life-like instances created by sampling uniformly at random elements from the corresponding original LOLIB instances.

The Taillard benchmark [Taillard 1993] is composed by random PFSP instances of different size. The processing times p_{ij} that characterize each instance are randomly sampled from a discrete uniform distribution $p_{ij} \in [1, 99]$.

REFERENCES

Hans Achatz, Peter Kleinschmidt, and J. Lambsdorff. 2006. Der corruption perceptions index und das linear ordering problem. *ORNews* 26, (2006), 10–12.

Tobias Achterberg. 2009. SCIP: Solving constraint integer programs. *Math. Program. Computat.* 1, 1 (2009), 1–41.

Paul E. Anderson, Timothy P. Chartier, Amy N. Langville, and Kathryn E. Pedings-Behling. 2021. The rankability of weighted data from pairwise comparisons. *Found. Data Sci.* 3, 1 (2021), 1.

Paul E. Anderson, Timothy P. Chartier, Amy N. Langville, and Kathryn E. Pedings-Behling. 2022. Fairness and the set of optimal rankings for the linear ordering problem. *Optim. Eng.* 23, 3 (2022), 1289–1317.

David Applegate, Ribert Bixby, Vasek Chvatal, and William Cook. 2006. Concorde tsp solver. http://www.tsp.gatech.edu/concorde

O Becker. 1967. Das helmstädtersche reihenfolgeproblem—die effizienz verschiedener näherungsverfahren. In *Computer Uses in the Social Sciences, Berichteiner Working Conference.*

Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. 2016. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940* (2016).

Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *Eur. J. Oper. Res.* 290, 2 (2021), 405–421.

Christian Blum and Andrea Roli. 2003. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Comput. Surv.* 35, 3 (2003), 268–308.

Thomas R. Cameron, Sebastian Charmot, and Jonad Pulaj. 2021. On the linear ordering problem and the rankability of data. *arXiv preprint arXiv:2104.05816* (2021).

Vicente Campos, Fred Glover, Manuel Laguna, and Rafael Martí. 2001. An experimental evaluation of a scatter search for the linear ordering problem. *J. Global Optim.* 21 (2001), 397–414.

Quentin Cappart, Didier Chételat, Elias Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. 2021a. Combinatorial optimization and reasoning with graph neural networks. *arXiv preprint arXiv:2102.09544* (2021).

Quentin Cappart, Thierry Moisan, Louis-Martin Rousseau, Isabeau Prémont-Schwarz, and Andre A. Cire. 2021b. Combining reinforcement learning and constraint programming for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, 3677–3687.

Josu Ceberio, Alexander Mendiburu, and Jose A. Lozano. 2015. The linear ordering problem revisited. *Eur. J. Oper. Res.* 241, 3 (2015), 686–696.

Xinyun Chen and Yuandong Tian. 2019. Learning to perform local rewriting for combinatorial optimization. *Adv. Neural Inf. Process. Syst.* 32 (2019).

Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259* (2014).

Nicos Christofides. 1976. The vehicle routing problem. Revue française d'automatique, informatique. *Recherche opérationnelle* 10, V1 (1976), 55–70.

Michel Deudon, Pierre Cournut, Alexandre Lacoste, Yossiri Adulyasak, and Louis-Martin Rousseau. 2018. Learning heuristics for the tsp by policy gradient. In *Proceedings of the International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Springer, 170–181.

Michael R. Garey and David S. Johnson. 1979. Computers and intractability. *The Journal of Symbolic Logic* (1979).

Andoni I. Garmendia, Josu Ceberio, and Alexander Mendiburu. 2022. Neural improvement heuristics for preference ranking. *arXiv preprint arXiv:2206.00383* (2022).

Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. 2019. Exact combinatorial optimization with graph convolutional neural networks. *Adv. Neural Inf. Process. Syst.* 32 (2019).

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. Generative adversarial networks. *Commun. ACM* 63, 11 (2020), 139–144.

Google. 2016. *Or-tools, Google Optimization Tools*. Retrieved from https://developers.google.com/optimization/routing

Martin Grötschel, Michael Jünger, and Gerhard Reinelt. 1983. Optimal triangulation of large real world input-output matrices. *Statistische Hefte* 25, 1 (1983), 261–295.

Jatinder N. D. Gupta and Edward F. Stafford Jr. 2006. Flowshop scheduling research after five decades. *Eur. J. Oper. Res.* 169, 3 (2006), 699–711.

Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (GELUS). *arXiv preprint arXiv:1606.08415* (2016).

Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Adv. Neural Inf. Process. Syst.* 33 (2020), 6840–6851.

John J. Hopfield and David W. Tank. 1985. "Neural" computation of decisions in optimization problems. *Biolog. Cybern.* 52, 3 (1985), 141–152.

Chaitanya K. Joshi, Quentin Cappart, Louis-Martin Rousseau, and Thomas Laurent. 2020. Learning TSP requires rethinking generalization. *arXiv preprint arXiv:2006.07054* (2020).

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. 2021. Highly accurate protein structure prediction with AlphaFold. *Nature* 596, 7873 (2021), 583–589.

Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. 2017a. Learning combinatorial optimization algorithms over graphs. *Adv. Neural Inf. Process. Syst.* 30 (2017).

Elias B. Khalil, Bistra Dilkina, George L. Nemhauser, Shabbir Ahmed, and Yufen Shao. 2017b. Learning to run heuristics in tree search. In *International Joint Conference on Artificial Intelligence*. 659–666.

Wouter Kool, Herke Van Hoof, and Max Welling. 2018. Attention, learn to solve routing problems! *arXiv preprint arXiv:1803.08475* (2018).

Yeong-Dae Kwon, Jinho Choo, Byoungjip Kim, Iljoo Yoon, Youngjune Gwon, and Seungjai Min. 2020. POMO: Policy optimization with multiple optima for reinforcement learning. *Adv. Neural Inf. Process. Syst.* 33 (2020), 21188–21198.

Yeong-Dae Kwon, Jinho Choo, Iljoo Yoon, Minah Park, Duwon Park, and Youngjune Gwon. 2021. Matrix encoding networks for neural combinatorial optimization. *Adv. Neural Inf. Process. Syst.* 34 (2021), 5138–5149.

Manuel Laguna, Rafael Marti, and Vicente Campos. 1999. Intensification and diversification with elite tabu search solutions for the linear ordering problem. *Comput. Oper. Res.* 26, 12 (1999), 1217–1230.

Wassily Leontief. 1986. *Input-output Economics*. Oxford University Press.

Luc Libralesso. 2020. *Anytime Tree Search for Combinatorial Optimization*. Ph. D. Dissertation. Université Grenoble Alpes.

Jiyin Liu and Colin R. Reeves. 2001. Constructive and composite heuristic solutions to the P//$\sum$ Ci scheduling problem. *Eur. J. Oper. Res.* 132, 2 (2001), 439–452.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

Lázaro Lugo, Carlos Segura, and Gara Miranda. 2021. A diversity-aware memetic algorithm for the linear ordering problem. *arXiv preprint arXiv:2106.02696* (2021).

Qiang Ma, Suwen Ge, Danyang He, Darshan Thaker, and Iddo Drori. 2019. Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning. *arXiv preprint arXiv:1911.04936* (2019).

Sahil Manchanda, Akash Mittal, Anuj Dhawan, Sourav Medya, Sayan Ranu, and Ambuj Singh. 2019. Learning heuristics over large graphs via deep reinforcement learning. *arXiv preprint arXiv:1903.03332* (2019).

Vinod Nair, Sergey Bartunov, Felix Gimeno, Ingrid Von Glehn, Pawel Lichocki, Ivan Lobov, Brendan O'Donoghue, Nicolas Sonnerat, Christian Tjandraatmadja, Pengming Wang, Ravichandra Addanki, Tharindi Hapuarachchi, Thomas Keck, James Keeling, Pushmeet Kohli, Ira Ktena, Yujia Li, Oriol Vinyals, and Yori Zwols. 2020. Solving mixed integer programs using neural networks. arXiv preprint arXiv:2012.13349 (2020).

Christos H. Papadimitriou and Kenneth Steiglitz. 1998. *Combinatorial Optimization: Algorithms and Complexity*. Courier Corporation.

Judea Pearl. 1984. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley Longman Publishing Co., Inc.

Gerhard Reinelt. 2002. Linear ordering library (LOLIB). Retrieved October 10, 2023 from http://comopt.ifi.uni-heidelberg.de/software/LOLIB/

Steven J. Rennie, Etienne Marcheret, Youssef Mroueh, Jerret Ross, and Vaibhava Goel. 2017. Self-critical sequence training for image captioning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 7008–7024.

Valentino Santucci, Marco Baioletti, and Alfredo Milani. 2015. Algebraic differential evolution algorithm for the permutation flowshop scheduling problem with total flowtime criterion. *IEEE Trans. Evolut. Computat.* 20, 5 (2015), 682–694.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. 2020. Mastering atari, go, chess and shogi by planning with a learned model. *Nature* 588, 7839 (2020), 604–609.

Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.

Eric Taillard. 1993. Benchmarks for basic scheduling problems. *Eur. J. Oper. Res.* 64, 2 (1993), 278–285.

El-Ghazali Talbi. 2021. Machine learning into metaheuristics: A survey and taxonomy. *ACM Comput. Surv.* 54, 6 (2021), 1–32.

Roy Tromble and Jason Eisner. 2009. Learning linear ordering problems for better translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 1007–1016.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30 (2017).

Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. 2015. Pointer networks. *Adv. Neural Inf. Process. Syst.* 28 (2015).

Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* 8, 3 (1992), 229–256.

Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang, and Andrew Lim. 2021. Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems* 33, 9 (2021), 5057–5069.

Wei Zhang and Thomas G. Dietterich. 2000. Solving combinatorial optimization tasks by reinforcement learning: A general methodology applied to resource-constrained scheduling. *J. Artif. Intell. Res.* 1 (2000), 1–38.

Received 26 October 2022; accepted 3 February 2024